

UI Automation Framework

thelallantop.com — Homepage Test Suite

Playwright + JavaScript | Desktop & Mobile Chrome

GitHub: github.com/srivashivansh/Lallantop-UI-automation

1. Project Overview

This document covers the complete UI automation framework built for the homepage of thelallantop.com. It is intended as both a technical reference and an interview preparation guide, explaining every design decision, tool choice, and implementation pattern used.

1.1 What We Built

- A Playwright-based UI automation framework in JavaScript
- Homepage test suite covering 16 test cases
- Two browser projects: Desktop Chrome (1280x720) and Mobile Chrome (Pixel 5 — 393x851)
- Page Object Model (POM) design pattern for maintainability
- Integrated with GitHub for version control and future CI/CD

1.2 Tech Stack

Tool / Technology	Version	Purpose
Playwright	v1.44+	Browser automation framework
JavaScript (JS)	ES6+	Programming language
Node.js	v18+	Runtime environment
VS Code	Latest	Code editor / IDE
GitHub	—	Version control & code hosting
Chrome Browser	Latest	Test execution browser

2. Why Playwright?

This is one of the most common interview questions. Here is a complete answer:

Feature	Playwright	Selenium
Auto-waiting	Built-in — waits for elements automatically	Manual waits required (Thread.sleep, explicit waits)
Speed	Faster — runs tests in parallel natively	Slower by default
Setup	One command: npm init playwright@latest	Complex WebDriver + browser driver setup
Cross-browser	Chromium, Firefox, WebKit (Safari)	All browsers + IE
Mobile emulation	Built-in device emulation	Requires Appium or extra config
Debugging	UI mode, trace viewer, Codegen tool	Limited native tools
Language support	JS/TS, Python, Java, C#	Java, Python, JS, C#, Ruby
Community	Growing fast (Microsoft-backed)	Large, mature ecosystem

Key Point: Interview answer: We chose Playwright because it has auto-waiting built in (reducing flaky tests), supports both desktop and mobile emulation natively, has excellent JavaScript support, and is significantly faster to set up compared to Selenium.

3. Project Structure

The project follows a clean, industry-standard folder structure:

```
lallantop-automation/  
  tests/  
    homepage.spec.js      <- All homepage test cases (16 tests)  
  pages/  
    HomePage.js           <- Page Object Model class  
  test-results/           <- Screenshots, videos, traces (auto-generated)  
  playwright-report/      <- HTML test report (auto-generated)  
  playwright.config.js     <- Framework configuration  
  package.json            <- Project dependencies and npm scripts  
  README.md              <- Setup and usage guide  
  .gitignore              <- Excludes node_modules, test-results
```

3.1 Why This Structure?

- tests/ — keeps test logic separated from page interaction logic
- pages/ — Page Object Model pattern; selectors live here, not in tests
- playwright-report/ — auto-generated; never committed to Git
- test-results/ — screenshots and videos only generated on failure

4. Playwright Configuration

The playwright.config.js file is the brain of the framework. It controls how tests run across both projects.

4.1 Key Configuration Settings

Setting	Value	Why
testDir	./tests	Where Playwright looks for test files
timeout	60000ms	Max time per test — increased for slow news site
navigationTimeout	60000ms	Max time for page.goto() — covers slow ad loading
actionTimeout	15000ms	Max time for clicks, fills, etc.
retries	1	Retry failed test once before marking as failed
workers	2	Run 2 tests in parallel
screenshot	only-on-failure	Screenshot taken only when a test fails
video	on-first-retry	Record video only on retry
trace	on-first-retry	Capture trace for debugging on retry

4.2 Two Projects: Desktop and Mobile

We configured two Playwright projects in a single config file, allowing us to run the same tests in both viewports with one command:

```
projects: [  
  {  
    name: 'desktop-chrome',  
    use: { ...devices['Desktop Chrome'], viewport: { width: 1280, height: 720 },  
    channel: 'chrome' },  
  },  
  {  
    name: 'mobile-chrome',  
    use: { ...devices['Pixel 5'], channel: 'chrome' } // 393 x 851  
  }  
]
```

Key Point: Interview answer: We use Playwright's projects feature to run the same test suite across desktop and mobile without duplicating code. Each project sets a different viewport and device profile. The channel: 'chrome' setting uses the locally installed Chrome browser rather than Playwright's bundled Chromium.

5. Page Object Model (POM)

5.1 What is POM?

Page Object Model is a design pattern where each page of the application has its own class file. This class contains all the selectors (locators) and actions for that page. Tests import and use this class rather than writing selectors directly in test code.

5.2 Why We Used POM

- Maintainability — if a selector changes on the website, you update it in one place (the page class), not in every test
- Readability — test code reads like plain English: `home.goto()`, `home.getTitle()`
- Reusability — multiple tests can use the same page actions without copy-pasting
- Separation of concerns — tests focus on what to verify; page classes focus on how to interact

5.3 Our HomePage.js Structure

```
class HomePage {
  constructor(page) {
    this.page = page;
    // Selectors defined once here
    this.logo = page.locator('a[href="https://www.thelallantop.com"] img').first();
    this.navShowsLink = page.locator('a[href*="/show"]').first();
    this.articleLinks = page.locator('a[href*="/post/"], a[href*="/video/"]');
  }

  async goto() {
    await this.page.goto('/', { waitUntil: 'domcontentloaded', timeout: 60000 });
  }
}
```

6. Test Suites & Test Cases

We organised 16 test cases into 3 logical suites. Each suite has a specific purpose and scope.

6.1 Suite 1 — Core Validations (runs on both Desktop & Mobile)

#	Test Name	What It Validates
1	Page loads with correct title	HTTP response + title contains 'Lallantop'

#	Test Name	What It Validates
2	No 404/500 errors	Response status < 400 using waitUntil: 'commit'
3	Logo is visible	Nav logo image is attached in the DOM
4	Navigation links are present	Home, Shows, Video links exist with exact absolute URLs
5	Multiple article links loaded	At least 5 article/video links present (content loaded)
6	Latest video section visible	Hindi heading visible after scrolling
7	Newsletter section present	SUBSCRIBE button attached in DOM
8	Meta description set (SEO)	meta[name='description'] is non-empty, length > 20
9	No broken images	img elements with naturalWidth = 0 <= 2
10	Open Graph title tag set	og:title contains 'Lallantop' (social sharing)
11	Article links are internal	hrefs point to thelallantop.com domain

6.2 Suite 2 — Desktop-specific (1280x720)

#	Test Name	What It Validates
12	Viewport is desktop width	viewport.width >= 1024px
13	Desktop nav bar visible	nav element rendered (not hamburger)
14	Partner links visible	Aajtak, Indiatoday etc. links in header
15	No horizontal scrollbar	body.scrollWidth <= window.innerWidth

6.3 Suite 3 — Mobile-specific (Pixel 5 — 393x851)

#	Test Name	What It Validates
12	Viewport is mobile width	viewport.width <= 768px
13	Logo visible on mobile	Nav logo image attached in DOM
14	Viewport meta tag set	meta[name='viewport'] contains width=device-width
15	No horizontal overflow	body.scrollWidth <= window.innerWidth + 5px tolerance
16	Links are tappable	Article link bounding box height > 20px

7. Locator Strategy

Choosing the right locator is one of the most important skills in UI automation. A bad locator makes tests flaky and hard to maintain.

7.1 Locator Priority (Best to Worst)

1. `getByRole()` — most resilient, based on accessibility semantics
2. `getByLabel()` — for form fields
3. `getByText()` / `getByPlaceholder()` — for visible text
4. CSS selectors with meaningful attributes — `href`, `alt`, `data-testid`
5. CSS selectors with class names — brittle, changes often
6. XPath — last resort, very brittle

7.2 Lessons Learned from Our Tests

Problem	Wrong Approach	Our Fix
Nav links not found	<code>a[href='/video']</code> (relative path)	<code>a[href='https://www.thelallantop.com/video']</code> (exact absolute URL from page snapshot)
Logo not visible	<code>img[alt='The Lallantop'].scrollIntoViewIfNeeded()</code>	<code>a[href='...'] img</code> — target parent nav element's child image
Wrong assertion type	<code>toBeVisible()</code> on CSS-hidden logo	<code>toBeAttached()</code> — logo is in DOM even if CSS hides it
Page load timeout	Default <code>waitUntil: 'load'</code>	<code>waitUntil: 'domcontentloaded'</code> — ads don't block tests

Key Point: Interview answer: We always inspect the actual page snapshot from failed tests to understand what the DOM really looks like, then write selectors based on that reality rather than assumptions. We prioritise href-based selectors for navigation elements because they are tied to the actual URL routing of the application.

8. Timeout & Wait Strategy

TheLallantop.com is a news website with heavy ad loading (Google DoubleClick, Adroll iframes). This caused multiple timeout failures. Here is how we handled them:

8.1 waitUntil Options

waitUntil Value	Waits Until	Used When
'commit'	First bytes received from server	HTTP status check test — response is all we need
'domcontentloaded'	HTML parsed, scripts deferred	Most tests — articles are in HTML; ads are separate
'load'	All resources loaded (default)	NOT used — heavy ads cause 30s+ timeouts

waitUntil Value	Waits Until	Used When
'networkidle'	No network requests for 500ms	NOT used — news sites always have background requests

8.2 Auto-waiting vs Explicit Waits

- Playwright auto-waits for elements to be actionable before clicking, filling, or asserting
- We use `toBeVisible({ timeout: 10000 })` only when we need to override the default
- We never use `page.waitForTimeout()` (sleep) — it makes tests slow and fragile
- We use `toBeAttached()` when an element exists in DOM but may be CSS-hidden

Key Point: Interview answer: We use `waitUntil: 'domcontentloaded'` for all our page navigations because `thelallantop.com` loads heavy ad iframes that never let the 'load' event fire within timeout. `domcontentloaded` fires as soon as the HTML is parsed, which is all we need since article content is server-rendered in the HTML, not loaded by ads.

9. Desktop vs Mobile Testing

9.1 How We Separate Desktop and Mobile Tests

We use `test.use({ viewport })` inside each test suite describe block. This overrides the project-level viewport for that entire suite:

```
test.describe('Desktop suite @desktop', () => {
  test.use({ viewport: { width: 1280, height: 720 } });
  // All tests here run at 1280px width
});
```

```
test.describe('Mobile suite @mobile', () => {
  test.use({ viewport: { width: 393, height: 851 } });
  // All tests here run at 393px width (Pixel 5)
});
```

9.2 Why `test.use()` Instead of `test.skip()`

Approach	Behaviour	Problem
<code>test.skip(isMobile)</code>	Test collected, then skipped	Shows 9 skipped in report; wastes runner time
<code>test.use(viewport)</code>	Correct viewport forced inside suite	Always passes correct size; no skips needed

Approach	Behaviour	Problem
grep in config	Filter by tag at project level	Unreliable in some Playwright versions; <code>test.use()</code> is safer

10. Assertion Strategy

10.1 Assertions Used in This Framework

Assertion	When to Use	Example in Our Tests
<code>toBeVisible()</code>	Element rendered & visible in viewport	Latest video section after scrolling
<code>toBeAttached()</code>	Element exists in DOM (may be hidden)	Logo image, nav links
<code>toContain()</code>	String contains expected substring	Page title contains 'Lallantop'
<code>toBeTruthy()</code>	Value is not null/undefined/empty	og:title meta tag has a value
<code>toBeGreaterThan()</code>	Number is above threshold	Article link count > 5
<code>toBeLessThan()</code>	Number is below threshold	Response status < 400
<code>toBeLessThanOrEqual()</code>	Number does not exceed limit	Broken images <= 2
<code>toBeFalsy()</code>	Value is false/0/null	No horizontal scroll

11. Debugging Failed Tests

11.1 Error Types We Encountered & Fixed

Error Type	Root Cause	Fix Applied
TimeoutError on <code>page.goto()</code>	<code>waitUntil: 'load'</code> blocked by ad iframes	Changed to <code>waitUntil: 'domcontentloaded'</code>
Element not found (<code>toBeAttached</code>)	Selector used relative href (<code>/video</code>) but site uses absolute URL	Used exact absolute URL in selector
Element hidden (<code>toBeVisible</code>)	Image has <code>loading='lazy'</code> , not rendered until scrolled	Used <code>toBeAttached()</code> + target parent nav element
<code>scrollIntoViewIfNeeded</code> timeout	CSS-hidden element can't be scrolled into view	Switched locator to visible parent element
Wrong viewport in tests	Project-level grep filter not applied reliably	Used <code>test.use({ viewport })</code> inside <code>describe</code> block

Error Type	Root Cause	Fix Applied
9 tests showing as Skipped	test.skip() marks tests collected-then-skipped	Replaced with test.use() viewport override

11.2 Debugging Tools in Playwright

- `--ui` flag — Opens visual test runner; see browser live, step through tests
- `--headed` flag — Runs tests with visible browser window
- `--slow-mo=1000` — Adds 1 second delay between actions so you can follow along
- `npm run report` — Opens HTML report with screenshots attached to failures
- Trace viewer — Records every action + DOM snapshot; open with `npx playwright show-trace`
- Page snapshot in error output — Shows actual DOM structure at failure point

Interview Tip: Interview tip: Always mention that you read the page snapshot from the error output to understand what the DOM actually looks like at the point of failure. This is much faster than guessing what went wrong.

12. Git Workflow

12.1 How We Used Git

7. Initialised repo locally with `git init`
8. Connected to GitHub remote: `git remote add origin https://github.com/srivashivansh/Lallantop-UI-automation.git`
9. Used Personal Access Token (PAT) for authentication — GitHub no longer accepts passwords
10. Every bug fix committed with a descriptive message describing what was fixed and why

12.2 Commit Message Convention We Followed

Prefix	Meaning	Example
fix:	Bug fix in existing test	fix: use <code>waitUntil domcontentloaded</code> for slow ad loading
feat:	New test or feature added	feat: add mobile viewport test suite
refactor:	Code restructured, no behaviour change	refactor: move all selectors into <code>HomePage POM</code> class
docs:	Documentation update	docs: add README setup instructions
chore:	Config or setup change	chore: increase <code>navigationTimeout</code> to 60s in config

12.3 What We Committed vs What We Excluded

- Committed: tests/, pages/, playwright.config.js, package.json, README.md, .gitignore
- Excluded via .gitignore: node_modules/, test-results/, playwright-report/, *.zip

13. npm Scripts Reference

Script	Command	When to Use
npm test	npx playwright test	Run all tests (desktop + mobile)
npm run test:desktop	npx playwright test --project=desktop-chrome	Run only desktop tests
npm run test:mobile	npx playwright test --project=mobile-chrome	Run only mobile tests
npm run test:headed	npx playwright test --headed	Run with browser window visible
npm run test:ui	npx playwright test --ui	Open visual UI test runner
npm run report	npx playwright show-report	Open HTML results report

14. Interview Questions & Answers

Interview Prep: This section contains the most commonly asked questions in SDET/QA automation interviews, with answers based on what we actually built.

Q1: What is Playwright and why did you choose it over Selenium?

Answer: Playwright is a Node.js automation framework by Microsoft. I chose it because it has built-in auto-waiting (no manual `Thread.sleep()`), supports desktop and mobile emulation natively in one config, runs tests in parallel out of the box, and has a much simpler setup than Selenium. For our news website which had heavy ad loading, the ability to configure `waitUntil` at the navigation level was critical.

Q2: What is the Page Object Model and why is it important?

Answer: POM is a design pattern where every page has a dedicated class containing all its selectors and actions. Tests import this class and call its methods. If a selector changes on the website, you update it in one place — the page class — instead of searching through all test files. It also makes tests readable because they use descriptive method names rather than raw CSS selectors.

Q3: How did you handle mobile and desktop testing in the same framework?

Answer: We configured two projects in `playwright.config.js` — `desktop-chrome` at 1280x720 and `mobile-chrome` using the Pixel 5 device profile at 393x851. For suite-specific viewport control, we

used `test.use({ viewport })` inside each describe block. This ensures the correct viewport is forced regardless of which project runs the suite.

Q4: What types of timeouts are there in Playwright and how did you configure them?

Answer: There are three levels: test timeout (max time for one test), navigationTimeout (max time for `page.goto()`), and actionTimeout (max time for clicks, fills). We set all three in `playwright.config.js`. For `thelallantop.com`, which loads heavy ad iframes, we increased navigationTimeout to 60 seconds and used `waitUntil: 'domcontentloaded'` so tests don't wait for ads to finish loading.

Q5: How do you debug a failing Playwright test?

Answer: First I read the error message and page snapshot in the output — this shows the actual DOM at the point of failure. Then I use `npm run report` to open the HTML report which includes screenshots. For deeper debugging I use `npx playwright test --ui` to step through the test visually, or `--headed --slow-mo=1000` to watch it run slowly. If the issue is complex, I enable trace recording and open it with `npx playwright show-trace`.

Q6: What is the difference between `toBeVisible()` and `toBeAttached()`?

Answer: `toBeVisible()` checks that the element is in the DOM AND rendered visibly — not hidden by CSS `display:none`, `visibility:hidden`, or zero opacity. `toBeAttached()` only checks that the element exists in the DOM — it can be CSS-hidden. In our tests, the Lallantop logo was in the DOM but hidden by CSS on initial load, so we used `toBeAttached()` to verify it exists without failing on visibility.

Q7: How do you handle flaky tests?

Answer: Flakiness usually comes from timing issues or bad locators. We address it by: (1) using Playwright's built-in auto-waiting instead of hardcoded sleeps, (2) choosing stable locators based on href or role rather than class names that change, (3) using `waitUntil: 'domcontentloaded'` for fast page loads, (4) setting retries: 1 in config so a genuinely flaky test gets one retry before failing, and (5) using `toBeAttached()` instead of `toBeVisible()` where appropriate.

Q8: How did you use Git in this project?

Answer: We initialised a local Git repo, connected it to a private GitHub repository, and pushed code using a Personal Access Token for authentication. We followed a conventional commit message format (fix:, feat:, refactor:) to make the history readable. We also set up a `.gitignore` to exclude `node_modules`, test results, and generated reports so only source code is tracked.

Q9: What is `waitUntil` in Playwright and which value did you use?

Answer: `waitUntil` controls what event Playwright waits for before considering a navigation complete. 'commit' — server responded. 'domcontentloaded' — HTML parsed. 'load' — all resources loaded (default). 'networkidle' — no network requests for 500ms. We used 'domcontentloaded' for most tests because `thelallantop.com`'s ad iframes prevent the 'load' event

from firing within 30 seconds, and used 'commit' for the HTTP status test where we only need the server response code.

Q10: How would you extend this framework to cover the video and article pages?

Answer: I would create new Page Object classes — VideoPage.js and ArticlePage.js — following the same pattern as HomePage.js. Then create new test files — video.spec.js and article.spec.js. For article tests, I'd navigate from the homepage by clicking the first article link, then validate the article page title, content body, author name, and breadcrumb. I'd also add API-level setup using Playwright's request context to seed test data if needed.

15. Next Steps & Roadmap

15.1 Immediate Next Steps

- Set up GitHub Actions CI/CD — run tests automatically on every git push
- Add article page test suite — click article, validate content loads
- Add search functionality tests — if site has a search bar
- Add video page tests — /video section

15.2 Advanced Improvements

- API + UI hybrid tests — use Playwright's APIRequestContext to set up test state faster
- Cross-browser coverage — add Firefox and WebKit (Safari) projects
- Visual regression testing — screenshot comparison using Playwright's toHaveScreenshot()
- Appium integration — extend the framework to cover Android and iOS native apps
- Allure report integration — richer reporting with history trends

15.3 GitHub Actions CI/CD (Sample Workflow)

```
name: Playwright Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with: { node-version: '18' }
      - run: npm install
      - run: npx playwright install chromium
      - run: npm test
```

16. Glossary

Term	Definition
Playwright	Microsoft's browser automation framework for Node.js
POM	Page Object Model — design pattern separating selectors from tests
Locator	Playwright's way of finding elements on a page
waitUntil	Controls what event <code>page.goto()</code> waits for before continuing
domcontentloaded	HTML is parsed; external resources (ads, images) may still be loading
toBeAttached()	Asserts element exists in DOM (may be hidden by CSS)
toBeVisible()	Asserts element exists AND is visually rendered in the viewport
Auto-waiting	Playwright automatically waits for elements before actions
Viewport	The visible area of the browser window
Device emulation	Simulating a mobile device's screen size and user agent
PAT	Personal Access Token — used for GitHub authentication instead of password
Flaky test	A test that sometimes passes and sometimes fails without code changes
Trace	Playwright recording of every action + DOM state for debugging
CI/CD	Continuous Integration/Deployment — running tests automatically on code changes

End of Document — Lallantop UI Automation Framework